

Understanding loc and iloc in Pandas

August 20, 2025

1 Introduction

This document explains the `loc` and `iloc` indexing methods in the Python pandas library, used for selecting data in DataFrames. It covers their definitions, syntax, purposes, and practical examples using a sample dataset.

2 Definitions

2.1 loc

`loc` is a **label-based** indexing method for accessing rows and columns in a DataFrame or Series by their index labels and column names.

- **Syntax:** `df.loc[row_labels, column_labels]`
- `row_labels`: Single label, list of labels, slice (e.g., `'start' : 'end'`), or boolean array.
- `column_labels`: Single column name, list of names, slice, or boolean array.
- **Key Feature:** Operates on explicit labels, making it intuitive for named or categorical indices.

2.2 iloc

`iloc` is an **integer-based** (position-based) indexing method for accessing rows and columns by their numerical positions (0-based indexing).

- **Syntax:** `df.iloc[row_indices, column_indices]`
- `row_indices`: Single integer, list of integers, slice (e.g., `start:end`), or boolean array.
- `column_indices`: Integer positions for columns.
- **Key Feature:** Ignores labels, using numerical positions like array indexing.

3 Why Use loc and iloc

These methods provide flexible ways to select data in pandas DataFrames, addressing different needs:

- **Data Selection and Subsetting:**

- `loc`: Ideal for selecting data using meaningful labels (e.g., 'Name', 'Jan').
- `iloc`: Useful for positional access, such as in loops or when labels are unknown.

- **Slicing and Conditions:**

- `loc`: Supports label-based slicing (inclusive of end) and boolean indexing.
- `iloc`: Supports integer-based slicing (exclusive of end).

- **Custom Indices:** `loc` excels with non-numeric indices; `iloc` treats DataFrames like matrices.

- **Readability vs. Performance:** `loc` is readable with labels; `iloc` is faster for position-based access.

- **Avoiding Ambiguity:** Explicitly distinguishes between label-based and position-based indexing.

4 Sample Dataset

To illustrate, consider the following pandas DataFrame created with Python:

```
1 import pandas as pd
2
3 data = {
4     'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Emma'],
5     'Age': [25, 30, 35, 40, 28],
6     'City': ['New York', 'Los Angeles', 'Chicago', 'Houston',
7             'Boston'],
8     'Salary': [50000, 60000, 75000, 80000, 55000]
9 }
10 df = pd.DataFrame(data, index=['a', 'b', 'c', 'd', 'e'])
```

This produces:

	Name	Age	City	Salary
a	Alice	25	New York	50000
b	Bob	30	Los Angeles	60000
c	Charlie	35	Chicago	75000
d	David	40	Houston	80000
e	Emma	28	Boston	55000

The DataFrame has index labels 'a' to 'e' and columns 'Name', 'Age', 'City', 'Salary'.

5 Examples of loc

Here are five examples demonstrating loc:

1. Single row by index label:

```
1 print(df.loc['a'])
```

Output:

Name	Alice
Age	25
City	New York
Salary	50000

Name: a, dtype: object

2. Multiple rows and specific columns:

```
1 print(df.loc[['a', 'c'], ['Name', 'Salary']])
```

Output:

	Name	Salary
a	Alice	50000
c	Charlie	75000

3. Range of rows using a slice:

```
1 print(df.loc['b':'d', 'Age':'City'])
```

Output:

	Age	City
b	30	Los Angeles
c	35	Chicago
d	40	Houston

4. Rows with a boolean condition:

```
1 print(df.loc[df['Age'] > 30, ['Name', 'Age']])
```

Output:

	Name	Age
c	Charlie	35
d	David	40

5. Single cell:

```
1 print(df.loc['e', 'City'])
```

Output:

Boston

6 Examples of iloc

Here are five examples demonstrating iloc:

1. Single row by position:

```
1 print(df.iloc[0])
```

Output:

```
Name      Alice
Age        25
City    New York
Salary    50000
Name: a, dtype: object
```

2. Multiple rows and columns by position:

```
1 print(df.iloc[[0, 2], [0, 3]])
```

Output:

```
      Name  Salary
a  Alice   50000
c  Charlie  75000
```

3. Range of rows and columns using slices:

```
1 print(df.iloc[1:4, 1:3])
```

Output:

```
      Age      City
b    30  Los Angeles
c    35    Chicago
d    40    Houston
```

4. Rows with a boolean array:

```
1 print(df.iloc[df['Age'].values > 30, [0, 1]])
```

Output:

```
      Name  Age
c  Charlie   35
d   David   40
```

5. Single cell by position:

```
1 print(df.iloc[4, 2])
```

Output:

Boston

7 Key Differences

- **Indexing Type:** `loc` uses labels; `iloc` uses integer positions.
- **Slicing:** `loc` includes the end label; `iloc` excludes the end position.
- **Use Case:** Use `loc` for label-based filtering; use `iloc` for positional access.
- **Performance:** `iloc` is faster for large datasets due to integer-based indexing.

8 Conclusion

`loc` and `iloc` are essential tools in pandas for data selection. `loc` is ideal for label-based access and filtering, while `iloc` suits position-based tasks and performance-critical applications. Understanding their differences ensures robust and efficient data manipulation.